1. 자기소개를 해주세요
안녕하세요 안양에 거주하고 있는 92년생 이진표 라고 합니다.
총 개발 경력은 6년6개월로, 첫 회사 솔루링크에서는 3년동안 EAI라는 솔루션과 웹 콘솔 시스템을 주로 개발했습니다.
현재 재직중인 회사 에스트래픽에서는 주로 한국도로공사 사업과 민자 영업소 요금징수 시스템의 운영단말을 개발해왔습니다.
저는 단순히 기능 구현 보다는 현업의 요구사항을 적극분석하고 반영하여 분석/설계를 통해 안정적인 시스템으로 구현하는데 강점이 있다고 생각합니다.
이런 경험을 바탕으로 넷마블 e-HR 사내시스템 개발에 기여하고 싶어 넷마블에 지원했습니다.

2. 왜 넷마블 e-HR 직무에 지원했나요?
저는 지금까지 현업의 요구사항 변화에 맞춰 시스템을 분석하고 개선하는 운영형 업무시스템을 중심으로 개발해왔습니다. 장애 대응, 기능 개선, 성능 최적화 등 실제 업무와 밀접한 기능을 수행하면서 단순 구현보다 업무 흐름을 이해하고 시스템으로 풀어내는 역량을 쌓아왔습니다.

넷마블 e-HR은 제가 지금까지 경험해온 도메인과는 다르지만, 결국 다양한 현업 요구사항을 이해하고 이를 시스템에 정확하게 녹여내야 하는 업무시스템이라는 점에서 본질은 같다고 생각합니다.

지금까지의 경험을 바탕으로 더 복잡한 업무 로직을 더 깊이 설계하고 구현하는 개발자로 성장하고 싶습니다.
우대사항을 보니까 DB에서 MSSQL, SPA에서 Angular빼고는 제가 다 경험해본 스킬들이라 적응도 빠르게 할 수 있다고 생각했습니다.

3. HR 시스템 경험이 없는데 괜찮다고 생각하나요?
네, 직접적으로 HR 시스템을 개발한 경험은 없습니다.
다만 지금까지 근무관리, 권한제어, 예외처리 등 정책과 규칙이 중요한 업무시스템을 지속적으로 개발해왔습니다.
이런 시스템들은 단순 기능 구현보다 정책, 제도, 예외 케이스를 정확히 이해하고 시스템에 반영하는 것이 중요하다고 생각합니다.
저는 새로운 도메인을 접했을 때 하드웨어 구성도, 소프트웨어 구성도, 업무 흐름도, 데이터 흐름도 등으로 구조화하여 이해해서 빠르게 적응하는 편입니다.
HR 시스템 역시 결국 제도와 프로세스를 정확하게 이해하고 이를 시스템화하는 업무라고 생각합니다.
그래서 HR 도메인 경험은 없더라도 빠르게 학습하고 충분히 기여할 수 있다고 생각합니다.

4. 본인이 생각하는 강점은 무엇인가요?
제 강점은 책임감, 운영 문제 해결 경험, 그리고 시스템 전체를 바라보는 시야입니다.
먼저 저는 맡은 일은 끝까지 책임지고 완수하려는 태도가 강합니다. 프로젝트를 수행하면서 일정 내 결과를 내기 위해 우선순위를 조정하고, 이슈가 발생해도 끝까지 해결하는 자세로 일해왔습니

다.
또한 운영 중인 시스템에서 발생하는 성능 이슈, 보안 취약점, 데이터 정합성 문제를 실무적으로 해결해본 경험이 많습니다. 그래서 단순 구현을 넘어 실제 운영 환경에서 필요한 안정성과 효율성을 함께 고려할 수 있습니다.

마지막으로 프론트엔드, 백엔드, DB, 배포까지 전체 흐름을 연결해서 볼 수 있는 시야가 있습니다. 광안대교 스마트톨링 프로젝트에서는 소프트웨어 아키텍처 설계부터 인증·인가, 대용량 처리, 배포 자동화까지 전반을 수행하며 시스템을 통합적으로 바라보는 역량을 키웠습니다.

5. OOP 기반 설계를 어떻게 해왔나요?
저는 OOP를 단순한 클래스 분리가 아니라, 책임을 명확히 나누고 변경에 유연하게 대응할 수 있게 만드는 설계 방식이라고 생각합니다.
스프링도 이런 객체지향 원칙을 잘 반영한 프레임워크라고 생각하고요.
실무에서는 화면, 비즈니스 로직, 데이터 접근 로직이 서로 섞이지 않도록 계층을 분리하는 데 신경을 많이 썼습니다.
인증, 권한, 주요기능처리 처럼 성격이 다른 기능들도 역할에 맞게 서비스와 공통 모듈로 나누어 설계했고, 예외가 많은 기능 역시 한 곳에 복잡하게 몰아넣기보다 책임별로 분리해서 유지보수하기 쉽게 구성했습니다.
운영단말 구축 시에도 공통 모듈과 처리 구조를 정리해 기능이 늘어나도 중복 구현을 줄이고 일관된 구조를 유지할 수 있도록 했습니다.

6. 가장 자신 있는 기술 스택은 무엇인가요?
가장 자신 있는 기술은 Java, Spring Boot, MyBatis, 그리고 Oracle·MySQL·MariaDB 같은 DBMS입니다.
프론트는 Vue 계열을 가장 많이 사용했고 React 경험도 있습니다.
특히 저는 단순 CRUD보다는 운영 시스템에서 많이 요구되는 권한 처리, 예외 케이스 반영, 대용량 조회 성능 개선, 정합성 검증 같은 부분에 강점이 있습니다.

7. Spring Boot를 쓰면서 좋았던 점은 무엇인가요?
Spring Boot는 설정이 간결하고 운영 환경까지 빠르게 구축할 수 있다는 점이 가장 좋았습니다.
기존 Spring Framework 대비 초기 설정과 의존성 관리가 단순해서 개발 생산성이 높았고, Actuator나 외부 설정 관리, 배포 편의성 면에서도 장점이 컸습니다.
실무에서는 인증, 배포, 로그, DB 연결 설정을 환경별로 분리해서 관리할 때 특히 효율적이었습니다.

8. MyBatis와 JPA는 어떻게 구분해서 사용하나요?
저는 복잡한 조회나 튜닝이 중요한 경우에는 MyBatis를 선호하고, 객체 중심 설계나 단순 CRUD 생산성이 중요한 경우에는 JPA가 유리하다고 생각합니다.

실무에서는 운영 시스템 특성상 복잡한 조회 조건, 프로시저 연계, 성능 튜닝 이슈가 많아서 MyBatis를 많이 사용했습니다.
반면 구조가 명확하고 엔티티 중심으로 관리할 수 있는 기능은 JPA도 충분히 좋은 선택이라고 생각합니다.
결국 기술 자체보다 시스템 특성과 운영 요구사항에 맞게 선택하는 게 중요하다고 봅니다.

9. SI에서 일했는데 사내 시스템 개발 환경에 잘 적응할 수 있을까요?
오히려 잘 맞을 수 있다고 생각합니다.
SI에서는 제한된 일정과 다양한 현업 요구사항 안에서 운영 가능한 시스템을 만들어야 했기 때문에, 요구사항 분석, 우선순위 판단, 영향도 검토, 운영 대응 같은 기본기가 많이 쌓였습니다.
다만 이제는 프로젝트 단위로 끝나는 환경보다, 하나의 시스템을 지속적으로 개선하고 구조적으로 고도화하는 환경에서 더 깊이 있게 일하고 싶습니다.
그래서 사내 핵심 시스템을 장기적으로 운영하는 e-HR 환경이 제 방향성과 잘 맞는다고 생각합니다.

10. 본이 넷마블 e-HR에 기여할 수 있는 부분은 무엇인가요?
저는 운영형 업무시스템에서 요구사항 분석, 설계(테이블구조정의서, 테이블매트릭스, 프로그램설계서-순서도,처리로직등), 개발, 성능개선, 보안 대응, 배포까지 연결해서 수행해온 경험이 있습니다.
특히 정책과 예외가 많은 시스템에서 정합성과 안정성을 확보하는 데 강점이 있습니다.
넷마블 e-HR에서도 인사제도 변경이나 신규 요구사항이 발생했을 때, 단순 기능 구현이 아니라 영향 범위를 구조적으로 분석하고 안정적으로 반영하는 방식으로 기여할 수 있다고 생각합니다.
또 현업과의 커뮤니케이션 경험도 많아서 개발과 업무 사이를 연결하는 역할도 잘할 수 있습니다.

이진표 기술경력서(총 경력 6년5개월~)

상세 경력사항 (총 경력: 3년)

2019. 04 ~ 2022.04 ㈜솔루링크 **기술지원부/사원**

회사정보: **EAI 솔루션 / 연 매출**: 6억 / **인원**: 7명

1. 주요 업무

- EAI 솔루션 개발, 테스트 및 유지보수
- 사용기술: Java, Spring, SpringBoot, Oracle, MySQL, Linux, Mybatis, Hibernate, JavaScript, JQuery, Vue.js, React.js, HTML, CSS, BootStrap

2. 수행 프로젝트 및 주요성과

1) 수협 Nextro 고도화 시스템 구축 (LG CNS / 수협은행, 2021.05 ~ 2022.01)

역할: EAI 시스템 구축 및 테스트

주요 수행 내용

- **Linux 환경 내 EAI 프로그램 설치, 개발, 테스트 및 시스템 모니터링 수행**

CentOS 기반 환경에서 EAI 프로그램의 설치, 개발, 테스트, 배포를 수행하였으며, 개발서버 및 테스트서버에서 충분한 검증을 거친 후 운영서버에 안정적으로 반영하였습니다.

- **고객(수협은행) 요구사항을 반영하여 자사 솔루션(URM) 커스터마이징 및 기능 개발**

URM은 고객이 실제 사용하는 웹 콘솔로, 마우스 기반 매핑 기능과 연계 관리 기능 등을 제공하는 시스템이었습니다. 고객 요구사항에 맞춰 주요 기능을 커스터마이징하고, UI/UX를 개선하여 **업무 편의성**을 높였습니다.

- **배치(Batch), 온라인(Online), 디퍼드(Deferred)등 다양한 거래 모델 개발 및 테스트환경 구성**

EAI 업무에서는 거래 특성에 따라 적합한 처리 방식을 적용하였습니다. 실시간 응답이 필요한 업무에는 API 및 Socket 기반의 온라인 거래 방식을, 대량 정산 및 이력 반영과 같이 즉시성이 낮은 업무에는 배치 방식을, 접수는 실시간으로 처리하되 후속 처리는 비동기로 수행해야 하는 업무에는 디퍼드 방식을 적용하여 개발 및 테스트를 수행하였습니다.

- spring batch 사용(@EnableBatchProcessing, Reader, Writer, Processor 등....)

- **개발, 테스트, 운영(Dev/Test/Prod) 서버 환경 구성 및 모니터링 체계 확립**

개발, 테스트, 운영 환경을 분리하여 단계별 검증 후 운영에 반영하는 체계를 갖추었으며, 서버 상태 및 시스템 동작 여부를 지속적으로 점검할 수 있도록 모니터링 체계를 운영하였습니다.

주요 성과

- 노후화된 은행 전산시스템의 대규모 교체 환경에서 안정적인 EAI 연계 인프라 구축완료
- 서버 이중화를 통한 시스템 고가용성 확보 및 무장애 운영 지원

was 2대 이중화, 톰캣 server.xml에서 앞단에서 들어오는 요청 구분 jvmRoute,
Cluster설정해서A 톰캣 장애 시 B 톰캣이 서비스를 지속할 수 있도록 이중화 및 클러스터 구성을 적용

---

2) 자사 EAI 웹 콘솔 솔루션(ISM) 고도화 (솔루링크, 2019.04 ~ 2020.05)

역할: 풀스택 개발 (Spring + Vue)

주요 수행 내용

- **기존 레거시(Adobe Flex) 화면을 Vue.js 기반의 모던 웹 환경으로 전환 개발(v2.4 → v2.5 고도화)**

Adobe Flex의 지원 중단에 대응하기 위해 기존 레거시 화면을 Vue.js 기반의 웹 환경으로 전환하였으며, 이를 통해 화면 구조를 현대화하고 유지보수성과 확장성을 개선하였습니다.

- **다양한 인터페이스 환경(DB to DB, DB to File, File to DB, File to File / 직접·간접 전송) 테스트 로직 수행**

EAI에서 사용하는 다양한 데이터 연계 방식을 테스트할 수 있도록 DB to DB, DB to File, File to DB, File to File 및 직접·간접 전송 구조에 대한 테스트 로직을 구현하였으며, 배치·온라인·디퍼드 거래 모델에 따른 실제 연계 시나리오를 구체화하였습니다.

- **온라인 거래 테스트를 위한 소켓(Socket) 통신 로직 및 DB 테스트용 SQL Client 개발**

온라인 거래 테스트를 위해 소켓 기반 전문 송수신 로직을 구현하였으며(System.arraycopy)
DB 검증 및 테스트 편의를 위해 java.sql.* 기반의 SQL Client 기능도 함께 개발하였습니다.

- **데이터베이스 테이블 동기화(Sync) 작업 및 Apache commons.net 라이브러리를 활용한 FTP 통신 기능 구현**

시스템 간 데이터 불일치 방지를 위한 테이블 동기화 로직을 구현 했고,
웹 환경에서 파일 송수신이 가능하도록 Apache Commons Net 라이브러리를 활용한 FTP 통신 기능을 개발하였습니다.

주요 성과

- Vue.js 전환을 통해 웹 콘솔의 UI/UX 및 유지보수성 대폭 개선

Adobe Flex 기반 레거시 화면을 Vue.js 기반으로 전환함으로써 사용자 편의성을 높이고, 향후 기능 개선 및 유지보수가 용이한 구조를 마련하였습니다.

- 복잡한 시스템 간 통합 전송 로직의 안정성 확보 및 시스템 변화에 빠른 대응이 가능한 확장

성 제공

다양한 데이터 연계 방식과 거래 모델을 테스트할 수 있는 구조를 마련하여 복잡한 통합 전송 로직의 안정성을 높였고, 시스템 환경 변화에도 유연하게 대응할 수 있는 기반을 확보하였습니다.

---

3) 사전 거래 관리 웹 콘솔 솔루션(URM) 전환 개발 (솔루링크, 2020.04 ~ 2021.04)

역할: 풀스택 개발 (Spring + React)

주요 수행 내용

- **Adobe Flex로 구현된 기존 URM 솔루션을 Spring Framework 기반의 React 환경으로 전면 재구축**

Adobe Flex 지원 중단에 대응하기 위해 기존 URM 솔루션의 화면 영역을 React 기반으로 전환하였으며, ISM 프로젝트의 Vue.js 전환 경험에 이어 URM은 React 기반으로 재구축하여 프론트엔드 기술 다변화와 유지보수성 향상을 함께 추진하였습니다.

- **필드, 필드그룹, 데이터, 데이터구조, 서비스, 인터페이스 등 핵심 거래 관리 컴포넌트 설계 및 개발**

거래 관리 화면의 핵심 메타 구조인 필드, 필드그룹, 데이터, 데이터구조, 서비스, 인터페이스 컴포넌트를 개발하였습니다.

각 구성 요소가 점진적으로 확장되는 계층형 구조를 기반으로 개발하여, 거래 정보 정의와 관리가 유기적으로 연결될 수 있도록 구현하였습니다.

- **Apache POI 라이브러리를 활용하여 대용량 거래 데이터 엑셀 다운로드 기능 구현**

Apache POI를 활용하여 대용량 거래 데이터를 엑셀로 다운로드할 수 있는 기능을 구현하였으며, XSSF, Sheet, Row, Cell 등을 활용해 거래 데이터를 동적으로 생성·가공하여 제공하였습니다.

XSSFWorkbook(사용)은 모든 데이터를 메모리에 올려 처리하므로 속도가 빠르지만 대용량 시 OutOfMemory가 발생할 수 있고, SXSSFWorkbook은 메모리를 주기적으로 임시 파일로 비워내며 대용량 엑셀을 처리하는 데 최적화되어 있습니다.

주요 성과

- 레거시 기술 전환 및 최신 프레임워크 도입으로 컴포넌트 재사용성 및 개발 생산성 향상

Adobe Flex 기반 레거시 구조를 React 기반으로 전환함으로써 화면 컴포넌트의 재사용성과 유지보수성을 높였고, 이후 기능 확장 및 개발 생산성 향상에 기여하였습니다.

- 고객사 요구에 유연하게 대응할 수 있는 아키텍처 기반 마련

거래 관리의 핵심 요소를 컴포넌트화하여 구조적으로 설계함으로써, 고객사 요구사항 변경이나 신규 기능 추가 시 보다 유연하게 대응할 수 있는 기반을 마련하였습니다.

---

4) 비즈니스 프로세스 통합 플랫폼(BW) 운영 및 모델링 (솔루링크, 2020.02 ~ 2021.02)
역할: 시스템 통합(EAI) 모델링 및 모니터링
주요 수행 내용

- **이기종 애플리케이션 통합을 위한 BW(Business Ware) 비즈니스 흐름 모델링 개발 및 배포**

BW(Business Ware) 플랫폼을 활용하여 이기종 시스템 간 연계를 위한 비즈니스 프로세스 흐름을 모델링하고, 이를 운영 환경에 배포하였습니다.

- **Linux 서버 환경에서 기동되는 BW 핵심 프로세스 및 주요 모델 상태 모니터링**

Linux 서버 환경에서 실행되는 BW 핵심 프로세스와 주요 비즈니스 모델의 기동 상태를 지속적으로 점검하고, 정상 동작 여부를 모니터링하여 운영 안정성을 지원하였습니다.

주요 성과

- 비즈니스 프로세스의 안정적인 운영 지원 및 거래 데이터 흐름의 가시성 확보

통합 플랫폼 내 프로세스와 모델을 안정적으로 운영할 수 있도록 지원하였으며, 시스템 간 거래 흐름을 추적·확인할 수 있는 기반을 마련하였습니다.

4. 이직사유

코로나19 이후 시장 환경 변화로 인해 기존 프로젝트들이 축소 및 지연되는 상황을 겪었습니다. 비록 어려운 환경이었으나, 저는 이에 안주하기보다 더 활발하게 프로젝트가 진행되는 안정적인 환경에서 제 역량을 발휘하고 싶었습니다. 회사의 사업 축소라는 수동적인 상황을 수용하기보다, 더 큰 시장에서 도전적으로 성과를 내기 위해 퇴사를 결심하게 되었습니다.

상세 경력사항 (총 경력: 3년5개월~)

2022. 09 ~ 에스트래픽㈜ **AX사업본부/대리**

회사정보: **교통(도로, 철도, AFC, 항공)관련 솔루션/ 연 매출**: 1461억 / **인원**: 247명(2025.09.30)

1. 주요 업무

- 요금징수시스템 개발, 한국도로공사 시스템 개발
- 사용기술: Java, Spring, SpringBoot, eGovFramework, Oracle, MySQL, MariaDB, Linux, Mybatis, JPA, JavaScript, TypeScript, Vue3.js, nuxt.js, Redis, Jenkins

2. 수행 프로젝트 및 주요성과

1) AI 영상학습시스템 구축 (한국도로공사, 2022.09 ~ 2024.01)

역할: 백엔드/프론트 개발

주요 수행 내용

- **학습 데이터셋 생성·삭제·통합 로직 구현 및 비동기 처리 최적화**

해당 프로젝트는 전체 일정이 약 6개월로 매우 짧았고, 제가 투입됐을 때는 이미 3개월이 지난 테스트 단계였지만 실제 모델 학습을 수행할 수 있는 환경은 제대로 갖춰져 있지 않았습니다.
그래서 우선 학습 데이터셋을 생성·삭제·통합하는 핵심 기능을 정비했고, 대량 데이터 처리 시 병목이 발생하는 부분은 비동기 방식으로 개선해 실제 학습이 가능한 환경을 만드는 데 집중했습니다.

- **JSON 기반 라벨링 확정 자동화(중복 ID, 좌표값, 정규식 검증 및 롤백 구조 구축)**

번호판 유형별로 최소 5만 건 이상의 학습 데이터가 필요했는데, 기존에는 라벨링 아르바이트 인력이 빠진 상태여서 내부 인력으로 직접 라벨링해야 했습니다. 이 방식이면 최소 5개월 이상 걸릴 상황이었습니다.
연구소에서 제공한 position 기반 핵심 로직을 시스템에 반영해 JSON 기반 자동 라벨링 배치를 구현했고, 이 과정에서 중복 ID, 좌표값 범위, 정규식 형식 등을 검증하도록 만들었습니다. 또한 오류 발생 시 이전 상태로 되돌릴 수 있도록 롤백 구조도 적용했습니다.
그 결과 약 3일 정도의 배치 처리로 자동 라벨링을 완료해 모델 학습에 성공적으로 반영할 수 있었고, 최종적으로 인식률 90% 이상 목표 달성에 기여했습니다.

- **유형별(일자/영업소/차로) 데이터 추출·검증 체계 수립으로 학습 데이터 누락 방지**

학습 데이터는 단순히 많이 모으는 것보다 품질이 중요했기 때문에, 정상·흐림·빛반사 같은 유형별로 데이터를 추출하고, 학습 대상이 아닌 데이터나 중복 데이터를 걸러내는 검증 체계를 만들었습니다.
검증 기준은 연구소 기준을 반영해 좌표값이 0.0~1.0 범위인지, 음수값이 존재하는지, position 선 정보가 4개인지 등을 체크하도록 구성했습니다. 이를 통해 잘못된 라벨 데이터가 학습에 들어가는 것을 방지했습니다.

- **3대 서버 분산 학습 구조 설계(Server Index 기반 병렬 처리)**

초기에는 1대 서버에서만 영상학습시켰는데, 엔진 하나 만드는 데 한 달 정도 걸릴 만큼 시간이 오래 걸렸습니다. 그래서 추가 AI 서버 2대가 도입되었고, 서버별 GPU 사양도 달랐습니다.
저는 학습 메뉴에서 어떤 GPU 서버를 사용할지 선택할 수 있도록 하고, 서버 관리 테이블에 Server Index를 두어 각 서버가 독립적으로 학습 작업을 수행하도록 분산 구조를 설계했습니다. 이를 통해 단일 서버 의존 구조를 벗어나 병렬 처리 기반의 학습 운영이 가능해졌습니다.

- **실시간 학습 현황 대시보드 고도화(잔여 시간·DB 사용량 시각화)**

당시 대시보드는 원그래프, 바그래프 같은 시각화 요소는 있었지만 실제 데이터 연계가 제대로 되지 않아 운영 관점에서 활용도가 낮았습니다.
그래서 Chart.js를 활용해 학습 진행 현황, 예상 잔여 시간, DB 사용량 등을 실제 데이터와 연계해 시각화했고, 운영자가 학습 상태를 직관적으로 확인할 수 있도록 개선했습니다.

- **Jenkins 기반 CI/CD 구축 및 JUnit 단위 테스트 환경 구성**

프로젝트 특성상 수정과 배포가 자주 발생했는데, 기존에는 배포 파일을 감독관에게 메일로 전달하고 배포 방법을 반복 설명해야 해서 비효율이 컸습니다.
그래서 VPN 정책 협의를 거쳐 배포 서버에 직접 접근 가능한 환경을 마련했고, Jenkins와 Git을 설치해 소스 push 시 자동 빌드 및 배포가 이뤄지도록 구성했습니다.
또한 JUnit 기반 단위 테스트 환경도 함께 정비해 배포 안정성을 높였습니다
Junit: @Test, assertThat(string).isEqualTo, assertEquals(A,B)...

주요 성과

- 데이터 정합성 자동 검증 체계 구축으로 학습 오류 사전 차단
- 서버 다중화로 학습 처리 확장성 확보

---

2) 통합영상관리 시스템 유지관리 (한국도로공사, 2022.09 ~ 2024.01)
역할: 기능 고도화 및 성능 개선
기존 운영 중인 시스템에 대해 장애 대응, 기능 개선 요청 반영, 성능 저하 구간 분석 및 최적화 등 유지관리 업무를 수행하였습니다.
주요 수행 내용

- **이륜차 진입 위반 영상조회 전용 화면 개발**

이륜차 진입 위반 데이터와 연계하여, 오토바이 관련 위반 영상만 별도로 조회할 수 있는 전용 화면을 개발하였습니다.

- **미인식 유형별 조회 조건 추가 및 쿼리 최적화**

미인식 유형 데이터가 많아 조회 성능 이슈가 발생하였고, 우선 조회 조건을 세분화하여 대상 범위를 축소하였습니다. 이후에도 성능이 충분하지 않아 조인 구조를 재검토하고, 단일 테이블 중심으로 데이터를 조회할 수 있도록 쿼리를 개선하여 성능을 최적화하였습니다.

- **IE 브라우저 호환성 문제 해결**

Chrome에서는 정상 동작하나 Internet Explorer 환경에서 특정 화면의 버튼이 표시되지 않는 문제가 발생하였습니다. 코드 분석 결과, 초기 Vue 기반 스크립트 영역에서 let 선언을 사용한 부분이 원인이었고, 이를 var로 변경하여 IE 환경에서도 정상적으로 화면이 표시되도록 개선하였습니다.

- **권한 기반 데이터 필터링 로직 적용**

본부/지사/영업소별로 조회 가능한 데이터 범위가 달라야 했기 때문에, 로그인 시 사용자 권한 정보를 세션에 저장하고 요청 처리 시 인터셉터를 통해 권한을 확인하여 권한별 데이터 조회 범위를 제어하는 로직을 적용하였습니다.

- **영상 비교조회 결과 엑셀 다운로드 기능 개선**

사용자 요청에 따라 기존에 없던 엑셀 다운로드 기능을 추가하였으며, Apache POI를 활용하여 영상 비교조회 결과 데이터를 엑셀 파일로 생성·다운로드할 수 있도록 구현하였습니다.

주요 성과

- **영상 조회 기능 고도화로 운영 효율 향상**

이륜차 전용 조회 화면, 미인식 유형별 조회 조건, 엑셀 다운로드 기능 등을 추가하여 운영자가 필요한 영상을 보다 빠르고 편리하게 확인할 수 있도록 개선하였습니다.

- **권한 기반 접근 통제로 데이터 보안 강화**

사용자 권한에 따른 데이터 조회 범위 제어 로직을 적용하여 조직별 접근 권한을 분리하고, 데이터 보안성과 관리 통제를 강화하였습니다.

---

3) 하이패스 원격유지관리 시스템 유지관리 (한국도로공사, 2022.09 ~ 2024.01)
역할: 기능 개선 및 보안 대응
기존 운영 중인 시스템에 대해 장애 대응, 기능 개선 요청 반영, 성능 저하 구간 분석 및 최적화 등 유지관리 업무를 수행하였습니다.
주요 수행 내용

- **차로설비현황 시각화 기능 개발**

상황실에서 차로별 설비 현황을 직관적으로 확인할 수 있도록 시각화 기능을 개발하였으며, Grafana를 활용해 하이패스 설비 고장 현황을 모니터링할 수 있는 화면을 구성하였습니다.

- **설비 데이터 엑셀 업·다운로드 기능 구현(POI 활용)**

설비 관련 데이터를 엑셀로 업로드 및 다운로드할 수 있는 기능이 없어, Apache POI를 활용하여 설비 데이터의 엑셀 입·출력 기능을 신규 구현하였습니다.

- **URL 직접 접속 차단(Web MVC allow method-get빠져있었음)**

특정 화면이나 기능이 정상적인 메뉴 흐름이나 권한 검증 없이 URL만 알고 있으면 직접 접근 가능한 구조가 없는지 점검했습니다.
조치할 때는 로그인 여부와 권한 체크를 강화하고, 필요한 경우 인터셉터나 필터 단에서 직접 접근을 막도록 했습니다.
즉, 사용자가 정해진 화면 흐름과 권한 범위 안에서만 접근할 수 있도록 통제한 겁니다.

- **JSON 데이터 노출 방지**

화면에서 사용하는 API 응답에 불필요한 내부 데이터나 디버그성 정보가 그대로 노출되지 않도록 점검한 겁니다.
필요한 필드만 내려주도록 응답 구조를 정리하고, 민감한 내부 정보나 불필요한 값은 제외하는 방식으로 조치했습니다.
결국 클라이언트에 전달되는 데이터를 최소화해서 정보 노출 위험을 줄인 것입니다.

- **AJP(Apache JServ Protocol) 버전 이슈**

  <Connector port="8009" protocol="AJP/1.3" redirectPort="8443" />

리포트 불러오는 화면에서 에러가 발생해서 로그를 보니까 AJP 관련 에러가 발생했습니다. 당시 AJP에 익숙하지 않아 먼저 개념과 설정 위치를 확인했고 Tomcat server.xml의 AJP Connector 설정을 점검했습니다. 근데 버전이 1.1로 돼있었습니다. 찾아보니 보통 1.3이더라구요. 그래서 1.3으로 수정하니까 에러는 잡혔습니다. 근데 이 시스템은 웹서버와 WAS가 분리된 구조가 아니라 WAR 형태로 단독 배포되는 방식이었기 때문에, AJP가 필수적이지 않다고 판단했습니다. 이에 따라 불필요한 AJP Connector는 비활성화하여 운영 구조를 단순화했고, 동시에 불필요한 내부 연동 포트 노출 가능성도 줄였습니다.

- **SSH 보안 조치 수행**

NGS 보안 프로그램이 업데이트가 안돼있는 문제를 발견해서 업데이트 조치 함

- **운영팀 대상 시스템 분석 및 기술 지원**

운영팀이 시스템 구조와 기능을 원활히 이해하고 활용할 수 있도록 시스템 분석 결과를 공유하고, 장애 및 운영 이슈에 대한 기술 지원을 수행하였습니다.

주요 성과

- 웹 보안 취약점 개선 및 운영 안정성 강화

보안 감사 대응 과정에서 주요 취약점을 개선하여 시스템 보안 수준을 높였고, 운영 중 발생하는 기능·장애 이슈에 대한 대응 체계를 강화하였습니다.

4) 광안대교 스마트톨링 요금징수시스템 운영단말 구축 (부산시설공단, 2024.02 ~ 2025.05)
역할: 전체 시스템 아키텍처 설계 및 개발 주도
주요 수행 내용

- Vue3 + Spring Boot 기반 SPA 아키텍처 설계

프론트엔드는 Vue 3 기반 SPA 구조로 설계했고, 백엔드는 Spring Boot 기반으로 설계했습니다. 기존에는 Vue2 경험이 있었지만, 그 시점에는 Vue 3가 안정화되어 있어서 Composition API 기반 구조를 적용했습니다. 빌드는 Vite, 상태관리는 Pinia, 비동기 통신은 Axios를 사용해 개발 환경을 구성했습니다. 백엔드는 설정모듈(DB, MVC등), 공통모듈, 컨트롤러-서비스-서비스임플-매퍼-xml까지 구조를 설계해 프론트와 역할을 분리했고, 이를 통해 화면 단과 비즈니스 로직 단의 책임을 명확히 나누어 유지보수성과 확장성을 높였습니다

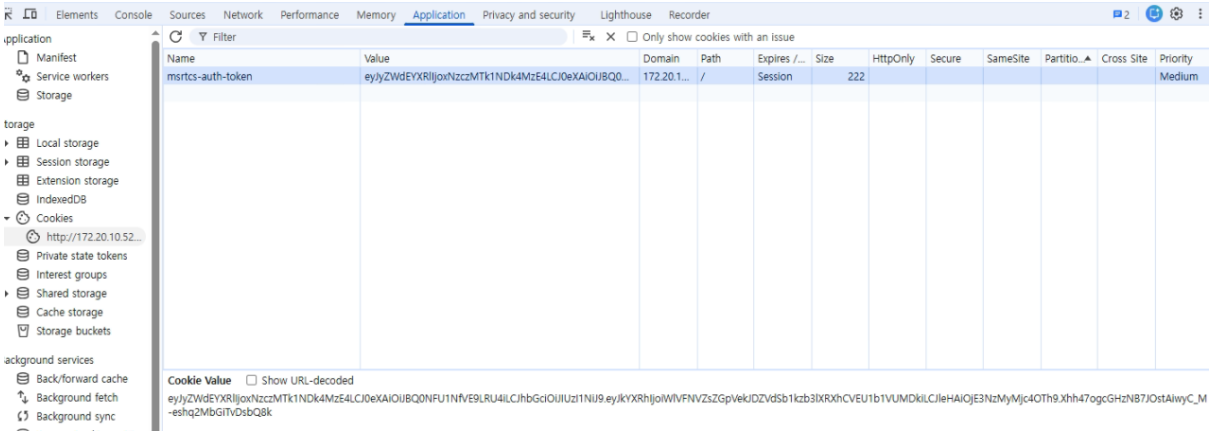
- 프론트/백엔드 공통 프레임워크 구축 및 표준화

프로젝트를 처음부터 진행하다 보니 공통으로 쓸것들을 먼저 만드는게 중요하다고 생각했습니다. 그래서 프론트와 백엔드에서 공통으로 사용할 수 있는 개발 표준과 프레임워크 구조를 먼저 정리했습니다. 백엔드에서는 DataSource, 매퍼설정, WebMVC설정(CORS), 예외 처리와 응답 구조를 공통화했고(단일 객체응답-MAP, 다중 객체응답- 리스트안에 MAP, ResponseEntity.*ok*), 프론트에서는 API 호출 방식,

```
export const request = async (method, url, param = {}, timeout = 2000000) => {
```

상태관리 방식, 공통 컴포넌트 사용 기준 등을 일관되게 맞췄습니다. 이를 통해 개발자마다 처리 방식이 달라지는 문제를 줄였고 eslint적용해서 코드스타일도 통일하고, 신규 기능 추가나 유지보수 시에도 빠르게 대응할 수 있는 기반을 마련했습니다.

pinia 상태관리 사용 경험
대메뉴-소메뉴 구조임.
화면에서 특정 소메뉴를 클릭하면 대메뉴 활성화-소메뉴 활성화 상태가 유지되야함.
또한 소메뉴 클릭시 탭으로도 생기는데 탭으로 눌러서 메뉴 이동도 가능함.
이때도 상태관리사용해서 구현



- JWT 기반 인증

JWT를 활용해 로그인 이후 요청을 인증하는 구조를 구현했습니다.
사용자가 화면에서 영업소 번호, 근무번호와, 비밀번호를 입력해서 로그인 합니다.
비밀번호는 암호화 되서 들어가구요
첫번째로 user테이블에 이 사용자 정보가 있는지 확인합니다.
사용자가 있으면 이제 토큰을 발급합니다.(엑세스토큰-HS256알고리즘사용)
발급한 토큰을 웹브라우저 쿠키에 저장합니다.
이때 httpOnly옵션은 true로 넣는게 중요합니다. 그래야만 웹브라우저에서 자바스크립트로 토큰 위변조가 불가능하게 됩니다.
자 그리고 이제 화면에서 뭔가 백단에 요청을할때 매 요청마다 토큰을 헤더에 담아서 보냅니다.
백단에서는 토큰을 받아 유효한지 검사를 하게 됩니다.

1. 스트링 형태로 전달받은 jwt를 파싱합니다.
2. 서버에 저장된 시크릿키를 가져와서 base64로 디코딩 후 서명키로 사용합니다.
3. jwt의 헤더와 페이로드를 서버에 저장된 시크릿 키를 가지고와서 해싱 합니다.
4. 해싱 결과가 토큰에 포함된 서명키와 일치하는지 검증 합니다.

jwt라이브러리를 사용하여 토큰이 있는지, 토큰이 만료되지 않았는지, 토큰이 변조되지 않았는지 등을 검사하게 됩니다.
만약 토큰에 문제가 있다면 바로 쿠키에 들어있는 토큰을 삭제하고 바로 로그아웃 페이지로 리다이렉트 해버립니다.
토큰에 문제가 없다면, 비로소 사용자의 요청 로직이 진행 됩니다.
기본적으로 토큰 만료시간은 9시간으로 했지만, 사용자가 계속 사용중이면 1시간 미만으로 남았을때 토큰을 재생성하는 로직도 추가했습니다.
이래야 업무 흐름이 끊기지 않고 계속 진행될 수 있기 때문입니다.

1. 왜 JWT를 사용했나요?

"로그인 이후 모든 요청에 대해 인증 상태를 확인해야 했기 때문에 JWT를 사용했습니다. 요청마다 토큰만 검증하면 되도록 구조를 단순화할 수 있었고, 인터셉터와 결합해 공통 인증 로직으로 처리하기에 적합했습니다."

2. JWT에는 어떤 정보를 넣었나요?
"토큰에는 사용자 식별값을 기준으로 필요한 최소 정보만 담았습니다. 특히 claim에는 사용자 번호를 그대로 넣지 않고 암호화한 값을 저장해서 직접 노출을 줄이도록 했습니다."

3. 토큰 검증은 어디서 했나요?
"Spring의 HandlerInterceptor를 사용해서 컨트롤러 진입 전에 토큰을 검증했습니다. preHandle()에서 Authorization 헤더나 일부 요청의 경우 쿠키 값을 확인하고, 유효하지 않으면 401을 반환하도록 처리했습니다."

4. 토큰 유효성은 어떻게 확인했나요?
"JWT 파싱 과정에서 서명과 만료시간을 함께 검증했습니다. 만료된 경우, 변조된 경우, 토큰이 없는 경우를 나눠 처리했고, 유효하지 않으면 요청을 차단했습니다."

5. 자동 재발급은 왜 넣었나요?
"업무 시스템 특성상 사용 시간이 길 수 있어서, 토큰이 갑자기 만료되면 불편이 컸습니다. 그래서 만료 시간이 1시간 이하로 남으면 서버에서 새 토큰을 발급해서 인증이 자연스럽게 이어지도록 했습니다."

6. 자동 재발급은 어떤 방식으로 했나요?
"기존 토큰의 claim에서 사용자 식별값을 꺼내고 복호화한 뒤, 같은 사용자 기준으로 새 JWT를 다시 생성했습니다. 이후 응답 쿠키에 새 토큰을 담아 갱신되도록 처리했습니다."

7. 세션 인증이 아니라 JWT를 쓴 이유는 뭔가요?
"이번 구조는 API 요청 단위로 인증을 처리하는 방식이 잘 맞아서 JWT를 선택했습니다. 인터셉터에서 공통으로 검증하기 쉬웠고, 프론트와 백엔드를 분리한 구조와도 잘 맞았습니다."

- 권한 체계 설계
  근무번호가 4자리인데 1로시작하면 관리자, 2로시작하면 일반근무자 3으로 시작하면 유지보수근무자 이런식으로 돼있습니다. 처음에 회원가입시 권한설정을 합니다. 근데 권한바뀌면 유지보수팀 연락와서 수동으로 처리했는데, 화면에 권한 바로 사용자들이 수정할 수 있도록 추가한거입니다.

- 근무관리 · 정산 · 송수신 핵심 로직 개발

요금징수 운영단말의 시작은 근무개시입니다. 그리고 마감 정산하고 최종적으로 도공과 송수신합니다. 즉, 영업소 단말의 일일 운영 흐름을 실제로 처리하는 핵심 업무 로직을 담당했습니다.

- 9만 건 이상 대용량 데이터 Virtual Scroll 적용
  우선 운영단말은 업무 특성상 페이징을 절대절대 하면 안되는 시스템입니다.

초기 개발시 리스트 형태로 표출되게 개발 했는데 잘 동작 했습니다.
그런데 데이터가 천건만 넘어가도 사용하지 못할 정도로 버벅임이 발생했습니다.
그래서 가상 스크롤 기술을 찾아봤고, 저희가 사용하고있는 UI 프레임워크 vuetify에서 가상 스크롤이 있어 적용을 했는데, 만건넘어가니까 또 버벅이더라구요. 이후에 vue에서 제공하는 가상 스크롤, element-plus 등...다양한 프레임워크에서 찾다가 모두 안됐는데 결국 Primevue라는 프레임워크에서 Virtual-scroller를 적용했는데 최대 9만건까지. 잘 동작 하더라구요..
어? 얘는 뭐가다르지? 라고 해서 좀까서 봤는데 특이하게 다른 프레임워크들은 단순히 item만 명시하면 됐는데 얘는 item-size라는걸 명시 하게 돼있더라구요...너비와 높이를 꼭 지정해야 하는데 이 기술 때문에 빨랐구나...생각했고.. 많은 테스트와..시행찾오를 격었었죠

운영중 장애나 오류 대응한 경험(트러블슈팅)
많은 장애/오류가 있었지만 음..지금 크게 기억 나는게

1. WEB/WAS OOM 자동 재시작
   백: PID읽어와 힘메모리 상태 진단
   jhsdb jmap(JDK설치시 기본적으로 사용 가능)
   JVM의 **Eden 영역과 Old 영역의 메모리 사용률**을 확인할 수 있게 되었습니다.
   가비지 컬렉션(GC)이 잘 일어나는지 운영 관점에서 들여다보는 시야
   프: Node.js환경의 Express로 웹서버 구축했고, nodejs내장객체 process.memoryUsage() 사용하여 메모리 헬스체크

2. DB 커넥션 풀로 인한 장애가 생각나네요.

• DB 튜닝 및 HikariCP 최적화
광안대교 스마트톨링 운영단말 프로젝트에서, 시스템이 평소와 다르게 간헐적으로 멈추는 문제가 있었습니다.
문제는 단순히 응답이 느린 수준이 아니라, 특정 시점이 되면 화면이 멈추고 백엔드에서도 정상적인 로그가 거의 남지 않는다는 점이었습니다.
처음에는 애플리케이션 로직 문제나 특정 API 장애로 의심했지만, 로그를 계속 확인해보니 공통적으로 **DB 커넥션 풀이 모두 소진된 상태에서 요청이 대기**하고 있다는 점을 발견했습니다.
이 상태가 되면 애플리케이션이 DB 연결을 얻지 못해서 정상 처리 흐름 자체가 막히고, 그 결과 로그도 기대한 만큼 남지 않는 구조였습니다.
그래서 DataSource와 HikariCP 설정을 직접 점검했고, 커넥션 풀 설정을 명확히 관리하도록 수정했습니다.
최대 커넥션 수, 유휴 커넥션 수, 커넥션 타임아웃을 명시적으로 설정했고, 종료 시점에 Active Connection과 Idle Connection 상태를 로그로 남기도록 해서 실제 커넥션 풀 상태를 확인할 수 있게 했습니다.

또한 DataSource가 종료될 때 명시적으로 close 되도록 처리해 자원 해제가 누락되지 않도록 했습니다.
이 경험을 통해, 장애 상황에서 로그가 적다고 해서 단순히 "로그가 안 찍혔다"로 볼 게 아니라, 오히려 **커넥션 풀이나 외부 자원 고갈처럼 더 아래 계층의 문제**를 먼저 의심해야 한다는 점을 배웠습니다.
또 운영 장애는 기능 코드만이 아니라 DataSource, 타임아웃, 자원 해제 같은 인프라 설정까지 함께 봐야 정확하게 해결할 수 있다는 점을 느꼈습니다.

- Jenkins 기반 CI/CD 구축 및 운영 서버 직접 구성

주요 성과
- 운영단말 시스템 신규 설계부터 배포까지 전 과정 수행
- 대용량 환경에서 성능 안정성 확보
- 배포 자동화 및 표준 아키텍처 기반 마련

---

5) 미시령 영업소 운영단말 고도화 (미시령 영업소, 2025.01 ~ 2025.10)
역할: 운영단말 기능 고도화
주요 수행 내용
- 심사 기능 고도화(하이패스 위반·사무실 심사 로직 개선)

미시령 영업소 운영단말 고도화 - 심사기능에서 redis 최초 적용
당시 사용자가 심사 팝업에서 한 건을 처리한 뒤 닫으면 목록을 다시 조회하는 구조였는데, 재조회에 약 30초 정도가 걸려 업무 흐름이 크게 끊기는 문제가 있었습니다.
처음에는 조인 구조와 조회 조건을 중심으로 쿼리 튜닝과 DB 구조 개선을 시도했지만 기대한 수준의 효과는 제한적이었습니다.
그래서 반복 조회되는 데이터의 특성을 보고 캐시 적용 가능성까지 포함해서 구조적으로 접근했습니다.
이 과정에서 단순히 DB만 보는 것이 아니라, 사용자의 조회 패턴을 분석해서 redis를 사용해보면 어떨까 고민했습니다.
그래서 Redis를 활용해 자주 조회되는 데이터를 메모리 기반으로 처리하도록 개선했고, 그 결과 기존에 약 30초 걸리던 조회 시간이 2~3초 수준으로 줄었습니다.
체감상으로는 약 10분의 1 수준까지 개선된 셈이어서, 사용자 만족도와 업무 처리 효율이 크게 좋아졌습니다.
이 경험을 통해 저는 성능 문제를 무조건 DB 튜닝만으로 접근하기보다, 데이터 조회 패턴과 사용자 업무 흐름까지 함께 보고 더 적절한 구조를 선택하는 것이 중요하다는 점을 배웠습니다.

- 특이 차량 케이스(보훈면제 등) 예외 처리 로직 보완
- 사용자 중심 UI 개선 및 운영 오류 최소화

주요 성과
- 심사 오류 발생 제로 수준 유지

- 복잡한 예외 케이스 처리 안정화

---

6) 방글라데시 N8 영업소 운영단말 시스템 구축 (방글라데시, 2025.07 ~ 2025.09)

역할: 해외 구축 프로젝트 개발

주요 수행 내용

- 현지 환경에 맞춘 운영단말 기능 설계 및 개발
- 요금 정산 및 통행 데이터 처리 로직 현지화
- 배포 및 운영 안정화 지원

주요 성과

- 해외 환경 적용 가능한 운영단말 시스템 구축 경험 확보

---

7) 대전천변 영업소 운영단말 고도화 (대전영업소, 2025.10 ~ 2025.12)

역할: 기능 개선 및 유지보수

주요 수행 내용

- 근무관리 기능 개발 및 정산 오류 수정
- TCS 근무마감 프로시저 오류 분석 및 개선
- 하이패스 강제근무 종료 기능 구현

주요 성과

- 정산 데이터 불일치 문제 개선 및 운영 안정성 확보

---

8) 서부간선 요금수납 운영 S/W 용역(PM) (케이알산업, 2025.06 ~)

역할: 유지보수 PM

주요 수행 내용

- 교통카드 BIN 변경에 따른 정산 불일치 대응
- 로그인 이력 관리 및 감사 대응 자료 제공
- 고지서 발송 로직 수정 및 배포 모니터링
- 분기 현장점검 및 유지보수 계약 관리

주요 성과

- 정산 데이터 오류 대응 및 운영 안정성 확보
- PM 역할 수행을 통한 프로젝트 관리 경험 축적

---

9) AI 기반 ITS 원격유지관리 시스템 구축 (한국도로공사, 2025.12 ~)

역할: 사업관리 및 개발 PL

주요 수행 내용

- 요구사항 정의서 작성 및 인터뷰 수행
- 과업대비표 및 WBS 수립
- 데이터 송수신 목록 정리 및 연계 구조 설계
- 착수보고 준비 및 사업관리 수행

주요 성과

- AI 기반 유지관리 체계 수립을 위한 초기 설계 및 사업관리 주도

3. 이직 사유

현재 재직 중인 SI 환경에서는 연차가 쌓임에 따라 직접적인 개발보다는 외주 인력 관리와 PM 업무로 직무가 전환되는 구조입니다. 저는 6년 동안 쌓아온 개발 역량을 바탕으로 단순히 프로젝트를 관리하는 관리자가 되기보다, 복잡한 문제를 기술적으로 해결하고 아키텍처를 설계할 수 있는 전문 개발자로 성장하고 싶다는 확신이 들어 이직을 준비하고 있습니다.

[활동 및 참고 링크]

Tech Blog | https://thefif19wlsvy.tistory.com

- 신입 시절부터 운영해 온 기술 블로그입니다.
- 단순한 코드 복사를 넘어, 새롭게 학습한 기술적 개념과 실무에서 겪은 문제 해결 과정(트러블슈팅)을 논리적으로 정리하며 성장의 발자취를 기록하고 있습니다.

GitHub (ODOC) | https://github.com/wlsvy19/ODOC

- 1일 1커밋(One Day One Commit)을 목표로 코딩을 일상화하기 위해 시작한 개인 저장소입니다.
- 단순한 습관 형성을 넘어, 새로운 기술을 테스트하고 실제 업무 효율을 높이는 데 필요한 유틸리티 프로그램을 직접 기획·구현하는 랩(Lab)실로 활용하고 있습니다.